



ClosingScript

This project may appeal to programmers who dislike CoffeeScript's reliance on significant indentation but otherwise appreciate the language for its simplicity and expressiveness.

ClosingScript addresses this issue by adding closing keywords to CoffeeScript syntax to terminate multi-line statements, making indentation no longer significant.

ClosingScript is a text preprocessor that regenerates significant indentation using these closing keywords. Its output can be piped directly into the official CoffeeScript compiler to make a one-step process: conversion → compilation → JavaScript.

A closing keyword consists of the slash followed by the name of its matching statement.

List of closing keywords:

/if, /unless, /while, /until, /for, /switch, /loop, /try, /class, /end (function)

Advantages:

- Whitespace-insensitive syntax.
- Robust code structure.
- Clear visual termination of multi-line statements.
- Safe copy-pasting.
- Safe auto-formatting.

Disadvantages:

- Slightly more verbose syntax.

Syntax

The syntax is identical to CoffeeScript syntax except for the addition of a closing keyword for multi-line statements. The list below is not exhaustive and is provided to give a general idea.

One-liners for **if**, **unless**, **while**, **until** and **for** are unchanged and do not need a closing keyword.

```
functionName = (parameters) -> | =>
```

```
  code
```

```
/end [optional following code]           # example of with following code: /end, timeout
```

```
do ->
```

```
  code
```

```
/end
```

```
if condition
```

```
  code
```

```
[else if condition
```

```
  code]
```

```
[else
```

```
  code]
```

```
/if
```

```
while condition [guard clauses]
```

```
  code
```

```
/while
```

```
for iterable [guard clauses]
```

```
  code
```

```
/for
```

```
loop
```

```
  code
```

```
/loop
```

```
switch [variable]
```

```
  when condition
```

```
    code
```

```
  [when condition then code]
```

```
  [else
```

```
    code]
```

```
/switch
```

```
class name
```

```
  code
```

```
/class
```

```
try
```

```
  code
```

```
[catch ...
```

```
  code]
```

```
[finally ...
```

```
  code]
```

```
/try
```

Syntax limitation

Multi-line constructs — statements, heredocs, and block comments — must stand alone on their lines.

This means that, for example, to assign a multi-line construct to a variable, the variable must be located on the preceding line.

```
variable =  
    ” heredoc on  
    multiple lines  
    ”
```

```
variable =  
    if condition  
        code  
    /if
```

Usage

The preprocessor outputs to *stdout*. To get JavaScript, the official CoffeeScript compiler must be installed.

Any error message with a line number from the preprocessor or the compiler will match the same line in a ***ClosingScript*** source.

Convert and compile to JavaScript in one step:

```
closing script.coffee | coffee -s -c > script.js
```

Get a converted copy of a source:

```
closing script.coffee > script.converted.coffee
```

Get a formatted copy of a source (only reindented consistently):

```
closing -f script.coffee > script.formatted.coffee
```

Usage with the Geany code editor

In Linux, this command can be added to the Build menu of the Geany code editor. Its converts and compiles in one step, with errors falling back at guilty lines in the editor window, as well as error messages from the preprocessor and the compiler are sent to the ‘Compiler’ message window. This conveniently makes the whole process as a single compiling step to the user.

```
set -o pipefail; closing "%f" | coffee -s -c > "%e".js    2> >(sed 's\[stdin\]/%f/' >&2)
```